

COURSE 3: VIEWS AND BLADE WITH LARAVEL

Dr. Sourour BELHAJ RHOUMA



Laravel

COURSE OUTCOMES

- ✓ Match each given URL to a specific process using routing
 - ✓ Group related processes using controllers
 - ✓ Retrieve data from an HTTP request using Request
 - ✓ Return content in text, HTML, JSON, etc. formats using Response
 - ✓ Integrate data into templates using Blade
 - ✓ Interact with the user using forms
 - ✓ Create, update, and track changes to a database system using migrations
 - ✓ Facilitate communication with a database using Eloquent
-

SUMMARY

1. Preliminaries
 2. Routing
 3. Controllers
 4. Requests
 5. Responses
 - 6. Views**
 7. Database
-

VIEW

1. Presentation
 2. Create and return a view
 3. Blade
 - Expression
 - Conditional statement
 - Iterative statement
 - Template heritage – parent
 - Template heritage – child
 - Inclusion of views
 - Forms
-

VIEW

- ✓ A view manages the layout and visual appearance of the interface.
 - ✓ Key benefits:
 - Clear separation between processing (logic) and presentation.
 - Facilitates collaboration between developers and designers.
 - ✓ Location: resources/views/
 - ✓ Can contain:
 - HTML for structure
 - CSS for styling
 - JavaScript for interactivity
 - PHP for dynamic content.
-

VIEW: CREATE AND RETURN A VIEW

✓Reto a simple view

```
return view('welcome');
```

✓Return a view from a function

```
return view('viewName');
```

```
return view('viewName')->with('variable', $value);
```

```
return view('viewName', ['cle' => $value]);
```

✓Return a view via a route

```
Route::view('/path', 'viewName');
```

```
Route::view('/path', 'viewName', ['data' => $value]);
```

Practical example:

```
// greeting.blade.php
```

```
<p>Hello {{ $firstname }} {{ $lastname }}! Are you {{ $age }} years  
old?</p>
```

```
// Dans le contrôleur
```

```
return view('greeting', compact('firstname', 'lastname', 'age'));
```

VUE: BLADE

- ✓ Laravel's integrated template engine
 - ✓ Template engine: a program that:
 - Facilitates the management of a template's dynamic content
 - Uses pseudo-code that's more accessible to the designer
 - Uses a cache
 - ✓ Blade interprets pseudo-code to:
 - Display the value of a PHP expression with `{{ }}`
 - Perform multiple tasks with directives:
 - `@if`, `@else`, `@switch` for conditions
 - `@for`, `@foreach`, `@while` for loops
 - `@include`, `@extends` for composition
-

VIEW: BLADE - EXPRESSION

✓ Expression display syntax `{{ expression }}`

✓ Examples:

- Simple variable: `{{ $age }}`
- Array element: `{{ $apprentiant['firstname'] }}`

✓ In greeting.blade.php, comment out the echo and add the following line below:

`Hello {{ $firstname }} . ' ' . $lastname }}! Are you {{ $age }} old?`

VUE: BLADE – CONDITIONAL STATEMENT

✓ Base structure @if

```
@if(condition)
    <!-- Content to display if the condition is true -->
@endif
```

✓ Structure @if with @else

```
@if(condition)
    <!-- Content to display if the condition is true-->
@else
    <!-- Content to display if the condition is false-->
@endif
```

✓ Structure @if with @elseif

```
@if(condition1)
    <!-- Content to display if the condition is true -->
}elseif(condition2)
    <!-- Content to display if the condition is true -->
@else
    <!-- Content to display if the condition is true-->
@endif
```

VIEW: BLADE – CONDITIONAL STATEMENT

✓ Directive **@isset**

@isset(\$variable)

<!-- Executed if \$variable is defined and not null-->

The variable contains: {{ \$variable }}

@endisset

✓ Directive **@empty**

@empty(\$collection)

<!-- Executed if \$collection is "empty" -->

(for an array: empty, for a string: empty, for null, etc.)

@else

<!-- Executed if \$collection is not empty -->

Number of elements: {{ count(\$collection) }}

@endempty

VIEW: BLADE – CONDITIONAL STATEMENTS

✓ Example:

```
@isset($user)
    <p>Welcome {{ $user -> name }}</p>
@endisset
```

```
@empty($table)
    <p>No items found</p>
@endempty
```

VIEW : BLADE – ITERATIVE STATEMENT

✓ **Loop @for:** → Génération de séquences.

```
@for($i = 0; $i < 10; $i++)  
    <p>Iteration n°{{ $i }}</p>  
@endfor
```

➔ Use: Traversal with an explicit counter (e.g. generation of numbered lists).

✓ **Loop @foreach:** → Displaying lists.

```
@foreach($users as $user)  
    <p>{{ $user->name }}</p>  
@endforeach
```

➔ Usage: Traversing an array or collection of objects.

VIEW: BLADE – ITERATIVE STATEMENT

✓ **Loop @forelse:** → Lists that may be empty.

```
@forelse($posts as $post)
    <article>{{ $post->title }}</article>
@empty
    <p>No items available.</p>
@endforelse
```

➔ Alternative to @foreach with automatic empty case management.

✓ **Loop @while:** → Repetitive conditional logic.

```
@while($condition)
    <p>Execute while the condition is true.</p>
@endwhile
```

➔ Usage: Repeat as long as a condition is true (eg incremental loading).

VIEW: BLADE – TEMPLATE INHERITANCE - PARENT

✓ Parent template in Blade

- Defines the content common to all child templates
- Creates sections that can be customized by children

✓ Section definition syntax:

✓ `@yield('sectionName', 'Default Content')`

ou

▪ `@section('nomSection')`

`<p> More elaborate default content</p>`

`...`

`@show`

➔NOTE:

- Use `@yield` for simple content
 - Prefer `@section/@show` for complex content
-

VIEW: BLADE –TEMPLATE INHERITANCE - CHILD

✓ Child template in Blade

- Must extend the parent: `@extends('path.to.parent')`
- Can override parent sections

✓ The child template overrides any section it wishes to populate by:

- **Overriding any default content**

```
@section('nomSection', 'New content')
```

- **Adding content with @parent**

```
@section('sectionName')
```

```
    New content
```

```
    @parent <!-- Keeps the parent's content -->
```

```
@endsection
```

VIEW: BLADE – INCLUSION OF VIEWS

✓ Blade Inclusion Directives

▪ Basic inclusion:

```
@include('view.path')
```

➔ The view has access to any variable in the surrounding template.

▪ With additional data:

```
@include('view.path', ['name' => $value])
```

➔ Inclusion with data passing

▪ Conditional inclusion:

```
@includeIf('view.path')
```

➔ Includes if the view exists

```
@includeWhen($condition, 'view.path')
```

➔ Includes if a condition

VIEW: BLADE – FORM

▪ Secure Blade Form: update,blade.php

```
<form action="{{ route('update', ['id' => $item['id']]) }}"
method="post">
@csrf <!-- CSRF protection required -->
@method('PUT') <!-- For HTTP methods other than GET/POST -->
<input name='lastname' value="{{ $item['name'] }}">
<input name='age' value="{{ $item['age'] }}">
<button type="submit">Update</button>
</form>
```

VIEW: BLADE – FORM

Corresponding routes:

```
// Display the form
Route::get('/update/{id}', [Controller::class, 'updateForm']);
// Process the submission
Route::put('/update/{id}', [Controller::class, 'update'])->
>name('update');
```

Key points:

Always use `@csrf` for POST forms

`@method` allows you to use PUT, DELETE, etc.

Use named routes (`route()`) for more flexibility
